

Unsupervised Learning Masterclass: Numerical Walkthroughs

A hand-calculable pedagogical guide tracing the core pillars of unsupervised learning.

1. K-Means Clustering: Step-by-Step Walkthrough

K-Means is an iterative, partition-based clustering algorithm. It groups data points into **K** distinct clusters by minimizing the distance between points and their respective cluster centers (centroids). It runs in a loop of **Assignment** (assigning points to the nearest centroid) and **Update** (moving centroids to the average center of their assigned points) until convergence.

The Sample Dataset

Imagine we have 4 retail customers with two features: Age and Spending Score (1-100):

- **Point A:** Age = 20, Spending = 80 (Young High Spender)
- **Point B:** Age = 22, Spending = 75 (Young High Spender)
- **Point C:** Age = 60, Spending = 20 (Senior Budget Shopper)
- **Point D:** Age = 62, Spending = 22 (Senior Budget Shopper)

We want to group them into **K = 2 clusters**.

Iteration 1: Starting the Loop

Step 1: Initial Centroid Placement

We randomly choose two points to act as our starting cluster centers:

- Centroid 1 (\$C_1\$): Placed at Point A (**20, 80**)
- Centroid 2 (\$C_2\$): Placed at Point C (**60, 20**)

Step 2: Distance & Cluster Assignment

We calculate the straight-line (Euclidean) distance from every point to both centroids: $\text{Distance} = \sqrt{(X_{\text{point}} - X_{\text{centroid}})^2 + (Y_{\text{point}} - Y_{\text{centroid}})^2}$

- **Point A (20, 80):** Dist to SC_1 is 0. Dist to SC_2 is 72.1. **Verdict: Cluster 1.**
- **Point B (22, 75):** Dist to SC_1 is $\sqrt{(22-20)^2 + (75-80)^2} = \sqrt{29} \approx 5.39$. Dist to SC_2 is 66.85. **Verdict: Cluster 1.**
- **Point C (60, 20):** Dist to SC_1 is 72.1. Dist to SC_2 is 0. **Verdict: Cluster 2.**
- **Point D (62, 22):** Dist to SC_1 is 71.61. Dist to SC_2 is $\sqrt{(62-60)^2 + (22-20)^2} = \sqrt{8} \approx 2.83$. **Verdict: Cluster 2.**

Step 3: Recalculate Centroid Positions (Centroid Update)

Centroids shift to the exact average center of their assigned members:

- **New Centroid 1 (SC_1')** (Average of A and B): Age = $\frac{20+22}{2} = 21$, Spending = $\frac{80+75}{2} = 77.5$. Coordinates: **(21, 77.5).**
- **New Centroid 2 (SC_2')** (Average of C and D): Age = $\frac{60+62}{2} = 61$, Spending = $\frac{20+22}{2} = 21$. Coordinates: **(61, 21).**

Iteration 2: Verifying Convergence

Step 4: Recalculate Distances & Assignments

We check if any points change their cluster labels when measured against the shifted centroids SC_1' (21, 77.5) and SC_2' (61, 21):

- **Point A (20, 80):** Dist to SC_1' is $\sqrt{(20-21)^2 + (80-77.5)^2} = 2.69$. Dist to SC_2' is 71.85. Remains in **Cluster 1.**
- **Point B (22, 75):** Dist to SC_1' is 2.69. Dist to SC_2' is 67.90. Remains in **Cluster 1.**
- **Point C (60, 20):** Dist to SC_1' is 69.90. Dist to SC_2' is $\sqrt{(60-61)^2 + (20-21)^2} = 1.41$. Remains in **Cluster 2.**
- **Point D (62, 22):** Dist to SC_1' is 69.11. Dist to SC_2' is 1.41. Remains in **Cluster 2.**

Since no points changed their cluster assignments, the algorithm has officially **converged!**

Customer	Age, Spend	Final Cluster	Demographic Segment
Point A	(20, 80)	Cluster 1	Young High Spenders
Point B	(22, 75)	Cluster 1	Young High Spenders

Customer	Age, Spend	Final Cluster	Demographic Segment
Point C	(60, 20)	Cluster 2	Senior Budget Shoppers
Point D	(62, 22)	Cluster 2	Senior Budget Shoppers

2. DBSCAN Clustering: Step-by-Step Walkthrough

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is one of the most important clustering algorithms because, unlike K-Means, you do not need to specify the number of clusters (K) in advance, and it is capable of discovering clusters of arbitrary shapes while cleanly filtering out outliers.

***The Core Philosophy:** Points that are in dense regions belong to the same cluster. Points that are isolated are treated as noise (outliers).*

Why DBSCAN? (K-Means vs. DBSCAN)

Consider a dataset with two clusters and one isolated point (outlier):

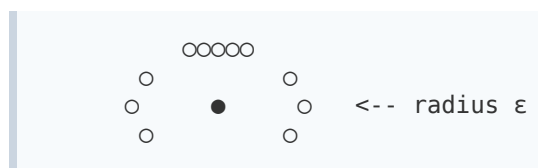


- **K-Means Problem:** If you choose $K = 2$, K-Means will force every point into one of the two clusters, dragging the cluster centers and assigning the isolated outlier ($\times$) to one of them.
- **DBSCAN Solution:** DBSCAN recognizes that the isolated point doesn't belong to any dense neighborhood and labels it as **Noise**.

DBSCAN Parameters

DBSCAN relies on two key parameters to define density:

1. **ϵ (Epsilon):** The radius around a point defining its search neighborhood. Imagine drawing a circle of radius ϵ around every point.
2. **MinPts:** The minimum number of points required inside the ϵ -radius (including the point itself) to call that region "dense".



Three Types of Points

DBSCAN classifies every data point into one of three categories:

- 1. **Core Point:** A point with at least `MinPts` neighbors inside its ϵ -neighborhood.

●

●

●

● [X] ●

●

●

●

(X has $\geq$ `MinPts` neighbors. Core Point!)

- 2. **Border Point:** A point that has fewer than `MinPts` neighbors, but falls within the ϵ -neighborhood of a core point.

●

●

●

● [X] ●

ε ○

(Border point: connected to Core point X, but not dense itself)

- 3. **Noise Point:** A point with too few neighbors and not reachable from any core point.

●

●

●

× (Too far away from any core point. Noise Outlier!)

Mathematical Definition

Let the ϵ -neighborhood of a point p be defined as: $N_{\epsilon}(p) = \{q \mid \text{distance}(p,q) \leq \epsilon\}$ A point p is a **Core Point** if: $|N_{\epsilon}(p)| \geq \text{MinPts}$ Otherwise, if p is in $N_{\epsilon}(c)$ for some core point c , it is a **Border Point**. Otherwise, it is a **Noise Point**.

The Sample Dataset

Imagine we have 7 points plotted on a 2D coordinate grid:

Point	X Coordinate	Y Coordinate
A	1	1
B	1	2
C	2	2
D	2	1

Point	X Coordinate	Y Coordinate
E	8	8
F	8	9
G	20	20

We configure DBSCAN with $\epsilon = 2$ and **MinPts** = 3.

Step-by-Step Numerical Scan

Step 1: Process Point A (1, 1)

1. Find all neighbors within $\epsilon = 2$ using Euclidean distance. Neighbors are {A, B, C, D}.
2. Count is 4. Since $4 \geq \text{MinPts} (3)$, **A is a Core Point**.
3. Start **Cluster 1**: $\{A\}$. Add A's neighbors $\{B, C, D\}$ to the expansion queue.

Step 2: Expand Cluster 1

We pull neighbors from the queue one by one and check if they can expand the cluster:

- **Process B (1, 2)**: Neighbors within radius 2 are {A, B, C, D} (count $4 \geq 3$). B is a Core Point! B joins Cluster 1.
1. Since its neighbors are already known, no new points are queued.
- **Process C (2, 2)**: Neighbors are {A, B, C, D} (count $4 \geq 3$). C is a Core Point! C joins Cluster 1.
- **Process D (2, 1)**: Neighbors are {A, B, C, D} (count $4 \geq 3$). D is a Core Point! D joins Cluster 1.

Cluster 1 expansion is complete. **Cluster 1** = {A, B, C, D}.

Step 3: Process Point E (8, 8)

1. Select E (next unvisited point). Neighbors are {E, F} (distance is 1.0).
2. Count is 2. Since $2 < \text{MinPts} (3)$, E is not a core point. It is temporarily marked as Noise.

Step 4: Process Point F (8, 9)

1. Select F. Neighbors are {E, F}. Count is 2. Since $2 < \text{MinPts} (3)$, F is not a core point.
2. Since neither E nor F is reachable from any core point, they remain unconnected.

Step 5: Process Point G (20, 20)

1. Neighbors of G is only {G}. Count is 1. Not a core point.
2. Marked as Noise.

Detailed Summary Chart

Point	Coordinates	Neighbors ($\epsilon=2$)	Point Class	Final Assignment
A	(1, 1)	4	Core Point	Cluster 1
B	(1, 2)	4	Core Point	Cluster 1
C	(2, 2)	4	Core Point	Cluster 1
D	(2, 1)	4	Core Point	Cluster 1
E	(8, 8)	2	Noise / Outlier	Noise (Isolated)
F	(8, 9)	2	Noise / Outlier	Noise (Isolated)
G	(20, 20)	1	Noise / Outlier	Noise (Outlier)

Deep Dive: Understanding "Expand Cluster" & Queue Expansion

The heart of DBSCAN is the **Expand Cluster** phase. It operates like a chain reaction (similar to **Breadth-First Search (BFS)** in graph theory). Once a starting core point is found, the cluster "spreads" to all reachable neighbors. If a neighbor is also a core point, it spreads the cluster to *its* neighbors, growing the cluster through connected dense regions.

***The Spilling Water Analogy:** Think of it like spilling water. Initially, water hits the starting core point. The water then spreads to all its neighbors. If those neighbors are also core points, they spread the water even further, creating a continuous flow until the water hits a boundary (border points) or runs out of connected dense points.*

The Queue-Based Chain Reaction Example

Let's trace this expansion meticulously using a queue. Suppose we have the following 8 points:

```

A  B  C
D  E  F
    G
          H

```

Coordinates: **A(1,3), B(2,3), C(3,3), D(1,2), E(2,2), F(3,2), G(3,1), H(10,10).**

We configure $\epsilon = 1.5$ and **MinPts = 3**.

1. Step 1: Pick Point A

A's neighbors within radius 1.5 are **{A, B, D, E}** (count $4 \geq 3$). A is a Core Point!

Start **Cluster 1 = {A}**. Initialize our search **Queue = [B, D, E]**.

2. Step 2: Pop B from Queue

B's neighbors are **{A, B, C, D, E, F}** (count $6 \geq 3$). B is a Core Point!

Add B to Cluster 1: **Cluster 1 = {A, B}**.

Since B is a Core Point, add its unvisited neighbors to the queue. **Queue = [D, E, C, F]**. (Note: C and F are newly discovered!).

3. Step 3: Pop D from Queue

D's neighbors are **{A, B, D, E}** (count $4 \geq 3$). D is a Core Point!

Add D to Cluster 1: **Cluster 1 = {A, B, D}**.

No new unvisited neighbors to add. **Queue = [E, C, F]**.

4. Step 4: Pop E from Queue

E's neighbors are **{A, B, C, D, E, F}** (count $6 \geq 3$). E is a Core Point!

Add E to Cluster 1: **Cluster 1** = {A, B, D, E}.

No new unvisited neighbors. **Queue** = [C, F].

5. Step 5: Pop C from Queue

C's neighbors are {B, C, E, F, G} (count $5 \geq 3$). C is a Core Point!

Add C to Cluster 1: **Cluster 1** = {A, B, D, E, C}.

Add C's unvisited neighbor G to the queue. **Queue** = [F, G].

6. Step 6: Pop F from Queue

F's neighbors are {B, C, E, F, G} (count $5 \geq 3$). F is a Core Point!

Add F to Cluster 1: **Cluster 1** = {A, B, D, E, C, F}.

No new unvisited neighbors. **Queue** = [G].

7. Step 7: Pop G from Queue

G's neighbors are {C, F, G} (count $3 \geq 3$). G is a Core Point!

Add G to Cluster 1: **Cluster 1** = {A, B, D, E, C, F, G}.

No new unvisited neighbors. **Queue** = [] (Queue is empty!).

The queue is empty, so expansion stops. Point H (10, 10) is too far away to ever be reached and is labeled as **Noise**.

Crucial Insight: Why do we still check D, E, etc., if they added no new points?

Because we don't know in advance whether they have additional neighbors! If the data had a branch extending down from D (like another point at (1, 0.5)), skipping D would mean missing that entire branch, leaving the cluster incomplete. Every point added to the cluster must be processed exactly once to guarantee we find every density-connected point.

DBSCAN Expansion Pseudocode

```
algorithm ExpandCluster(dataset, point, neighbors, cluster_id, eps, min_pts) is
    assign point to cluster_id
    seeds := queue(neighbors)

    while seeds is not empty do
        current_point := seeds.pop()

        if current_point is unvisited then
            mark current_point as visited
            current_neighbors := FindNeighbors(dataset, current_point, eps)

            if length(current_neighbors) ≥ min_pts then
                seeds.push_all(current_neighbors)

        if current_point is not assigned to any cluster then
```

```
assign current_point to cluster_id
```

Difference Between K-Means and DBSCAN

Feature	K-Means Clustering	DBSCAN Clustering
Number of Clusters	Must specify K in advance	Discovered automatically based on density
Core Mechanism	Uses centroids (cluster means)	Uses density (neighbor count within ϵ radius)
Outlier Handling	Forces every point into a cluster (sensitive)	Naturally detects and isolates noise points
Cluster Shapes	Assumes spherical/globular clusters	Finds arbitrary, complex, and winding shapes
Key Parameters	K (Number of Clusters)	ϵ (Search Radius) & MinPts (Density Limit)

Real-World Applications

- **Credit Card Fraud Detection:** Grouping normal transactions together while isolating fraudulent transactions as noise.
- **GPS Trajectory Analysis:** Clustering coordinates to identify frequently visited hotspots or routes.
- **Earthquake Epicenter Clustering:** Detecting seismic fault lines by grouping dense clusters of earthquake coordinates.
- **Customer Segmentation with Outliers:** Clustering core customer segments without letting spam accounts or extreme spenders distort the groups.

Interview Spotlight: "What is DBSCAN?"

"DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is an unsupervised density-based clustering algorithm that groups points located in close proximity (dense regions) while labeling isolated points in sparse regions as noise. It requires two main parameters: ϵ (neighborhood radius) and MinPts (density threshold). Unlike K-Means, DBSCAN does not require specifying the number of clusters in

advance, can discover clusters of arbitrary shapes (like rings or spirals), and is highly robust to outliers, which it automatically identifies."

3. PCA Dimensionality Reduction: Step-by-Step Walkthrough

Principal Component Analysis (PCA) simplifies high-dimensional data by projecting it onto a lower-dimensional subspace (e.g., compressing 2D down to 1D) while keeping the maximum amount of original spread (variance) of the shape.

The Sample Dataset

Imagine we have 3 points plotted on a 2D coordinate grid:

- **Point A:** (1, 2)
- **Point B:** (3, 4)
- **Point C:** (5, 6)

These points form a diagonal line. We want to compress them into a single **1D coordinate**.

Step-by-Step Mathematical Compression

Step 1: Mean Centering

We shift the data so its average center rests exactly at (0, 0).

1. Calculate averages: Mean X = $\frac{1+3+5}{3} = 3$, Mean Y = $\frac{2+4+6}{3} = 4$.
2. Subtract the means:
 - **Point A'** = $(1 - 3, 2 - 4) = \mathbf{(-2, -2)}$
 - **Point B'** = $(3 - 3, 4 - 4) = \mathbf{(0, 0)}$
 - **Point C'** = $(5 - 3, 6 - 4) = \mathbf{(2, 2)}$

Step 2: Compute the Covariance Matrix

Using the sample covariance formula (dividing by $N-1 = 2$):

1. $\text{Cov}(X, X) = \frac{(-2)^2 + 0^2 + 2^2}{2} = \mathbf{4}$
2. $\text{Cov}(Y, Y) = \frac{(-2)^2 + 0^2 + 2^2}{2} = \mathbf{4}$
3. $\text{Cov}(X, Y) = \frac{(-2 \times -2) + (0 \times 0) + (2 \times 2)}{2} = \mathbf{4}$

Our Covariance Matrix (\$\$\$) is:
$$S = \begin{pmatrix} 4 & 4 \\ 4 & 4 \end{pmatrix}$$

Step 3: Find Eigenvalues & Eigenvectors

1. Solve $\det(S - \lambda I) = 0$:
$$\det \begin{pmatrix} 4 - \lambda & 4 \\ 4 & 4 - \lambda \end{pmatrix} = 0$$

$\implies \lambda^2 - 8\lambda = 0$ This yields: $\lambda_1 = 8$ (PC1, captures 100% of variance) and $\lambda_2 = 0$ (PC2).

2. Solve for the eigenvector $\mathbf{v}$ for $\lambda_1 = 8$: $\begin{pmatrix} -4 & 4 \\ 4 & -4 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = 0 \implies x = y$ Normalizing this vector gives: $\mathbf{v} = \begin{pmatrix} 0.7071 \\ 0.7071 \end{pmatrix}$

Step 4: Projection to 1D

We calculate the dot product of centered coordinates with $\mathbf{v}$: $\text{Projected Coordinate} = (X_{\text{centered}} \times 0.7071) + (Y_{\text{centered}} \times 0.7071)$

- **Point A' (-2, -2):** $(-2 \times 0.7071) + (-2 \times 0.7071) = \mathbf{-2.83}$
- **Point B' (0, 0):** $(0 \times 0.7071) + (0 \times 0.7071) = \mathbf{0.00}$
- **Point C' (2, 2):** $(2 \times 0.7071) + (2 \times 0.7071) = \mathbf{2.83}$

Original 2D Point	Centered 2D Point	Compressed 1D Coordinate	Information Retained
Point A (1, 2)	(-2, -2)	-2.83	100% (No loss)
Point B (3, 4)	(0, 0)	0.00	100% (No loss)
Point C (5, 6)	(2, 2)	2.83	100% (No loss)

4. Isolation Forest Outliers: Step-by-Step Walkthrough

Isolation Forest isolates anomalies directly by recursively slicing the data space with random lines. Because anomalies are sparse and far away from the crowd, they get isolated in very few slices (shallow tree depth), whereas normal points require many slices to separate.

The Sample Dataset

Imagine we are auditing Credit Card Transactions with two features: Spend Amount (\$) and Distance from Home (km). We have 4 transactions (3 normal, 1 outlier):

- **Point A:** Spend = \$20, Distance = 2 km (Normal)
- **Point B:** Spend = \$25, Distance = 5 km (Normal)
- **Point C:** Spend = \$30, Distance = 3 km (Normal)
- **Point X:** Spend = \$500, Distance = 80 km (Anomaly Outlier)

Step-by-Step Space Slicing

Round 1: The First Split

1. Pick random feature: The algorithm randomly chooses **Spend Amount**.
2. Find range: The minimum spend is \$20 (Point A) and maximum is \$500 (Point X).
3. Pick random split value: It picks a random number between \$20\$ and \$500\$. Let's say it picks **\$150**.
4. Divide the dataset:

- Left Branch (Spend < \$150): Contains Point A, Point B, Point C.
- Right Branch (Spend ≥ \$150): Contains Point X.

✨ **Point X is now completely alone. It is isolated in just 1 split! (Path Length = 1)**

Round 2: Splitting the Left Branch

We look only at the remaining group {A, B, C} on the left branch:

1. Pick random feature: It randomly chooses **Distance from Home**.
2. Find range: Min = 2 km, Max = 5 km.
3. Pick random split value: It picks a random number between 2 and 5. Let's say it picks **4 km**.
4. Divide the remaining dataset:

- Left Sub-Branch (Distance < 4 km): Contains Point A (2 km) and Point C (3 km).
- Right Sub-Branch (Distance ≥ 4 km): Contains Point B (5 km).

🌟 **Point B is now completely alone. It is isolated after 2 splits! (Path Length = 2)**

Round 3: Splitting the Last Group

Only Point A and Point C are left together:

1. Pick random feature: It chooses ****Spend Amount****.
2. Find range: Min = \$20, Max = \$30.
3. Pick random split value: It randomly picks ****\$26****.
4. Divide the dataset:
- Left Sub-Branch (Spend < \$26): Contains Point A (\$20).
 - Right Sub-Branch (Spend ≥ \$26): Contains Point C (\$30).

🌟 **Point A and Point C are now both isolated. (Path Length = 3 for both)**

Data Point	Transaction Details	Splits to Isolate (Path Length)	Verdict
Point X	Anomaly (\$500, 80km)	1 split (Very Shallow)	 HIGH RISK FRAUD FLAG
Point B	Normal (\$25, 5km)	2 splits	 Approved (Normal)
Point A	Normal (\$20, 2km)	3 splits (Deep)	 Approved (Normal)
Point C	Normal (\$30, 3km)	3 splits (Deep)	 Approved (Normal)

5. Apriori Association Rules: Step-by-Step Walkthrough

The Apriori algorithm is used for Market Basket Analysis to discover purchase relationships between products. It operates on the principle that: *If an itemset is frequent, then all of its subsets must also be frequent*. This allows the algorithm to prune (discard) unpopular item combinations early, saving massive amounts of computation. It evaluates rules using **Support**, **Confidence**, and **Lift**.

The Sample Dataset

Imagine our supermarket database has exactly 3 customer receipts (transactions):

- **Receipt 1 (T1):** {Bread, Butter}
- **Receipt 2 (T2):** {Bread, Butter, Milk}
- **Receipt 3 (T3):** {Milk}

We configure Apriori with a **Minimum Support of 50%** (must appear in ≥ 2 receipts) and a **Minimum Confidence of 60%**.

Step-by-Step Association Mining

Step 1: Candidate Single Items (\$C_1\$)

We list individual items and count their support:

- **{Bread}**: 2 receipts (T1, T2) → Support = $\frac{2}{3} = \mathbf{66.7\%}$ (Passes!)
- **{Butter}**: 2 receipts (T1, T2) → Support = $\frac{2}{3} = \mathbf{66.7\%}$ (Passes!)
- **{Milk}**: 2 receipts (T2, T3) → Support = $\frac{2}{3} = \mathbf{66.7\%}$ (Passes!)

All items become **Frequent 1-Itemsets (\$L_1\$)**.

Step 2: Candidate Item Pairs (\$C_2\$)

We combine items from \$L_1\$ into pairs and count support:

- **{Bread, Butter}**: 2 receipts (T1, T2) → Support = $\frac{2}{3} = \mathbf{66.7\%}$ (Passes!)
- **{Bread, Milk}**: 1 receipt (T2) → Support = $\frac{1}{3} = \mathbf{33.3\%}$ (✗ **Pruned! Below 50%**)
- **{Butter, Milk}**: 1 receipt (T2) → Support = $\frac{1}{3} = \mathbf{33.3\%}$ (✗ **Pruned! Below 50%**)

Only **{Bread, Butter}** becomes a **Frequent 2-Itemset (\$L_2\$)**.

Step 3: Rule Generation & Metric Audits

We extract rules from our frequent pair {Bread, Butter}:

- **Rule A: $[Bread] \rightarrow [Butter]$**
 - Support (Popularity): Support of {Bread, Butter} = $\mathbf{66.7\%}$
 - Confidence (Reliability): $\frac{\text{Support}(Bread, Butter)}{\text{Support}(Bread)} = \frac{2}{2} = \mathbf{100\%}$ (Passes $\geq 60\%$!)
 - Lift (Strength): $\frac{\text{Confidence}}{\text{Support}(Butter)} = \frac{1.00}{0.667} = \mathbf{1.5x}$ (Positive correlation!)
- **Rule B: $[Butter] \rightarrow [Bread]$**
 - Support: $\mathbf{66.7\%}$
 - Confidence: $\frac{\text{Support}(Bread, Butter)}{\text{Support}(Butter)} = \frac{2}{2} = \mathbf{100\%}$ (Passes!)
 - Lift: $\frac{1.00}{0.667} = \mathbf{1.5x}$

Discovered Rule	Rule Popularity (Support)	Rule Reliability (Confidence)	Strength Multiplier (Lift)	Retail Recommendation
$[Bread] \rightarrow [Butter]$	66.7%	100%	1.5x	Place Bread and Butter on the same aisle to drive convenience buys.
$[Butter] \rightarrow [Bread]$	66.7%	100%	1.5x	Bundle them together in a "Breakfast Special" package promotion.